

Lec 08

Application layer: Web & HTTP

Computer Networks (CSE 232)

Rinku Shah

These slides are inspired and adapted from:

1. *Computer Networking: A Top-Down Approach*
6th edition, Jim Kurose, Keith Ross, Pearson, 2017



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY
DELHI

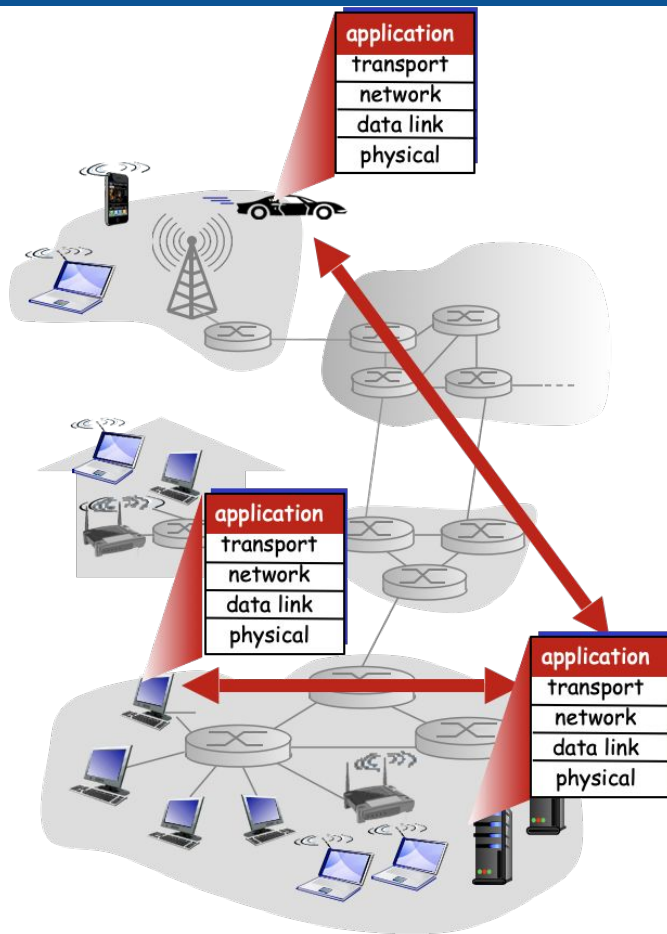
Chapter outline

- Want to create a network application?
 - Network application architectures (Client-server and P2P)
 - What is an application layer protocol? What services does it require from the transport layer?
- Web and HTTP
- FTP
- Electronic mail
 - SMTP, POP3, IMAP
- DNS
- P2P applications
- Socket programming with UDP and TCP (COVERED EARLIER)

Some network applications

- E-mail
- Web
- Text messaging
- Remote login
- P2P file sharing
- Multiuser network games
- Streaming stored videos (Youtube, Netflix, ...)
- Voice over IP (Skype, ...)
- Real-time video conferencing (Zoom, ...)
- Social networking
- Search
- ...

Creating a network application



Write programs that:

- Run on end systems and communicate over network
- E.g., Web server software

No need to write software for network core devices

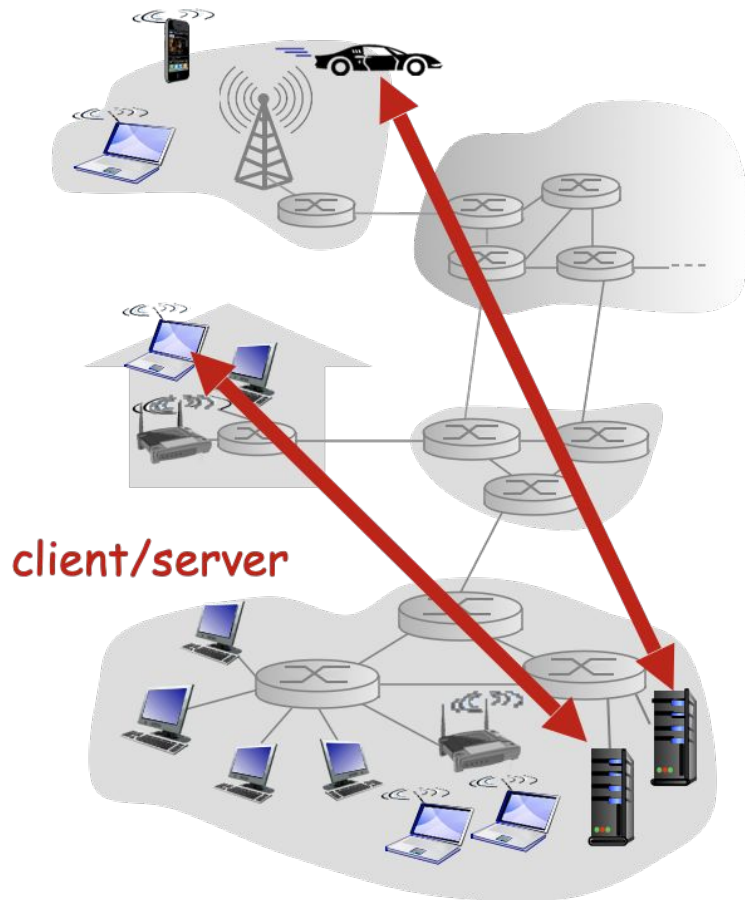
- They do not run user applications

Application architectures

Possible application structure

- Client-server
- Peer-to-Peer (P2P)

Client-server architecture



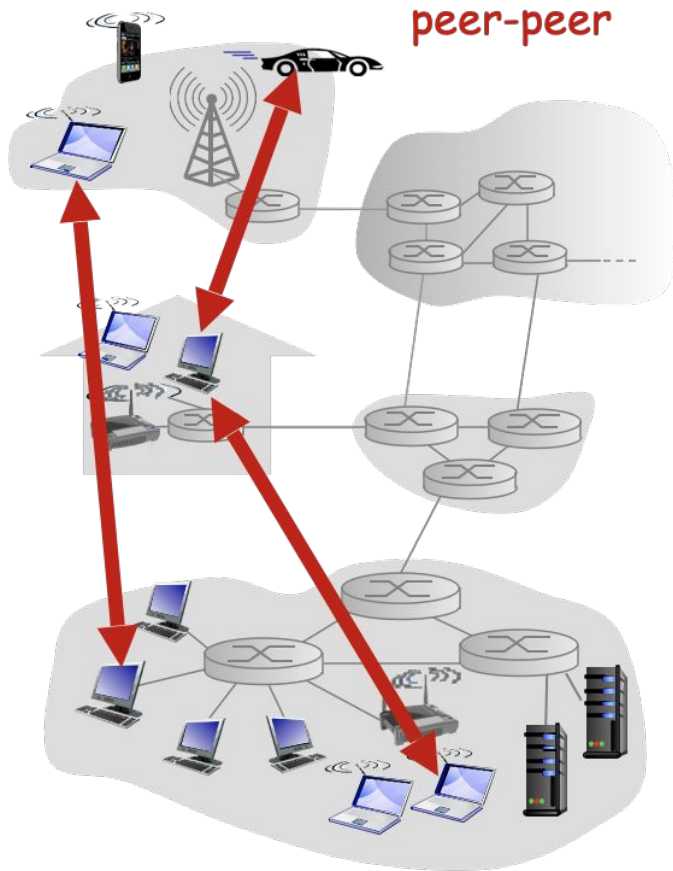
Server

- Always-ON host
- Has permanent IP address
- Scalability managed by data centers

Client

- Maybe intermittently connected
- May have dynamic IP address
- Clients do not directly communicate with each other

Peer-to-Peer architecture



- Arbitrary end-systems directly communicate
- No always-ON device
- Peers request service from other peers & provide service to other peers
 - Self scalability: New peers bring new service capacity as well as service demands
- Peers are intermittently connected and change IP addresses
 - Complex management

Addressing the communicating processes

To receive messages, process should have an identifier

- Host device is identified by a unique IP address
- Is IP address enough to receive messages?
 - Multiple processes may be running on the same host
- Receiving process identifier includes IP address and port number associated with the receiving host
 - Communication endpoint is called a 'Socket'
 - Connection identifier is a 4 tuple
 - source IP, source port, dest IP, dest port

Application layer protocol defines

- Types of messages exchanged
- Message syntax
- Message semantics
- Rules

Application protocol

- Open protocols
- Proprietary protocols

Services required from the transport layer

- Data integrity
- Timing
- Throughput
- Security

Common applications and their requirements

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
text messaging	no loss	elastic	yes and no

Internet transport protocol services

TCP service:

- ❖ *reliable transport* between sending and receiving process
- ❖ *flow control*: sender won't overwhelm receiver
- ❖ *congestion control*: throttle sender when network overloaded
- ❖ *does not provide*: timing, minimum throughput guarantee, security
- ❖ *connection-oriented*: setup required between client and server processes

UDP service:

- ❖ *unreliable data transfer* between sending and receiving process
- ❖ *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: why bother? Why is there a UDP?

Internet apps: application & transport protocols

application	application layer protocol	underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

**For background on HTTP, persistent connections,
non-persistent connections,**

Refer to Kurose slides and in-class notes

Types of HTTP messages: HTTP Request & HTTP response

HTTP request message

- ASCII readable format
- Example HTTP GET request header

request line (GET, POST, HEAD commands) →

header lines →

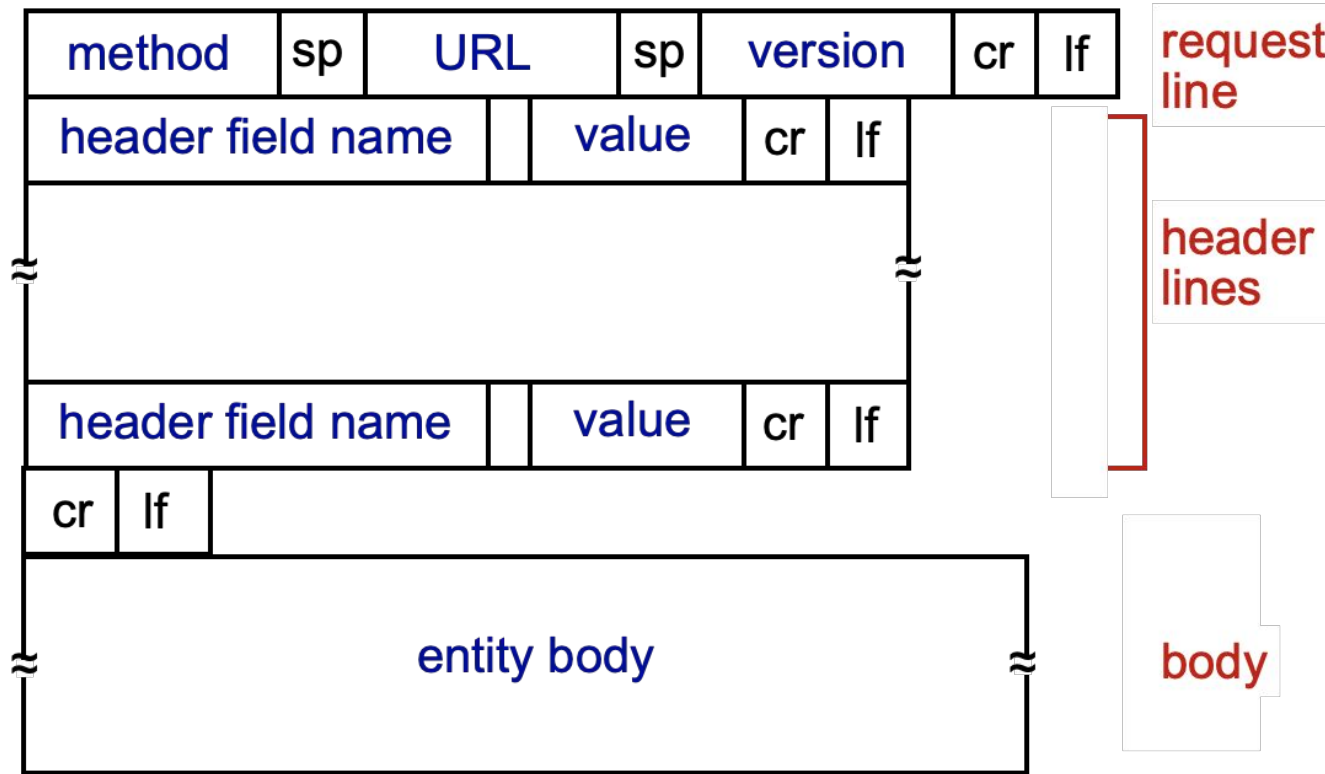
```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return character
line-feed character

carriage return, line feed
at start of line indicates
end of header lines

Body of the message

HTTP request message format



Uploading form input

POST method

Input is uploaded to server in entity body

GET method (aka, URL method)

Input is uploaded in the URL field of request line

`www.somesite.com/coursesearch?course=computer-networks§ion=B`

Other HTTP request messages

- **GET**
- **POST**
- **HEAD**
 - Requested headers (only) would be returned, if specified URL were requested using an HTTP GET method
- **PUT**
 - Uploads a new file object to the server
 - File from entity body is uploaded to path specified in the URL
- **DELETE**
 - Deletes file specified in the URL field

HTTP/1.0

- GET
- POST
- HEAD

HTTP/1.1

- GET
- POST
- HEAD
- PUT
- DELETE

HTTP response message

status line (protocol
status code status phrase)

header
lines

data, e.g., requested
HTML file

HTTP/1.1 200 OK

Date: Tue, 08 Sep 2020 00:53:20 GMT

**Server: Apache/2.4.6 (CentOS) OpenSSL/
1.0.2k-fips PHP/7.4.9 mod_perl/2.0.11
Perl/v5.16.3**

Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT

ETag: "a5b-52d015789ee9e"

Accept-Ranges: bytes

Content-Length: 2651

Content-Type: text/html; charset=UTF-8

\r\n

data data data data data ...

HTTP response status codes

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. netcat to your server

```
% nc -c -v info.cern.ch 80
```

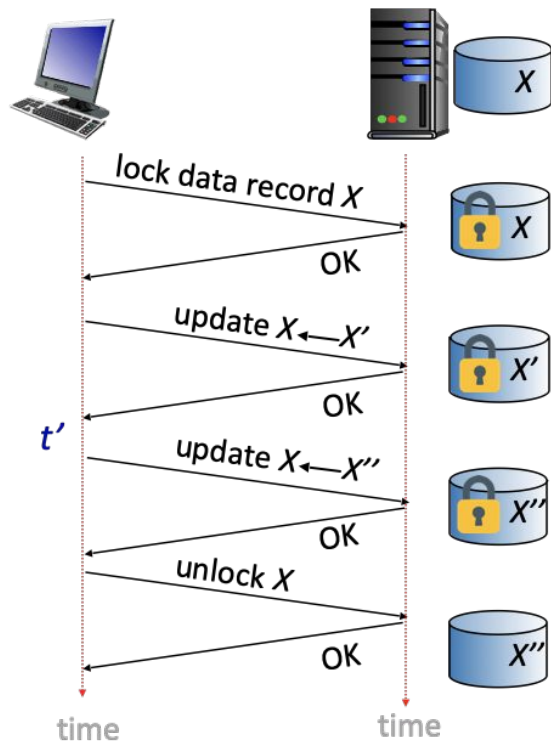
2. type in a GET HTTP request

```
GET /hypertext/WWW/TheProject.html HTTP/1.1  
Host: info.cern.ch
```

3. Look at the response message sent by HTTP server!
 - a. You may also use Wireshark to look at captured HTTP request/response

Example application that requires stateful protocol

Client makes two changes to “X”, or NONE at all

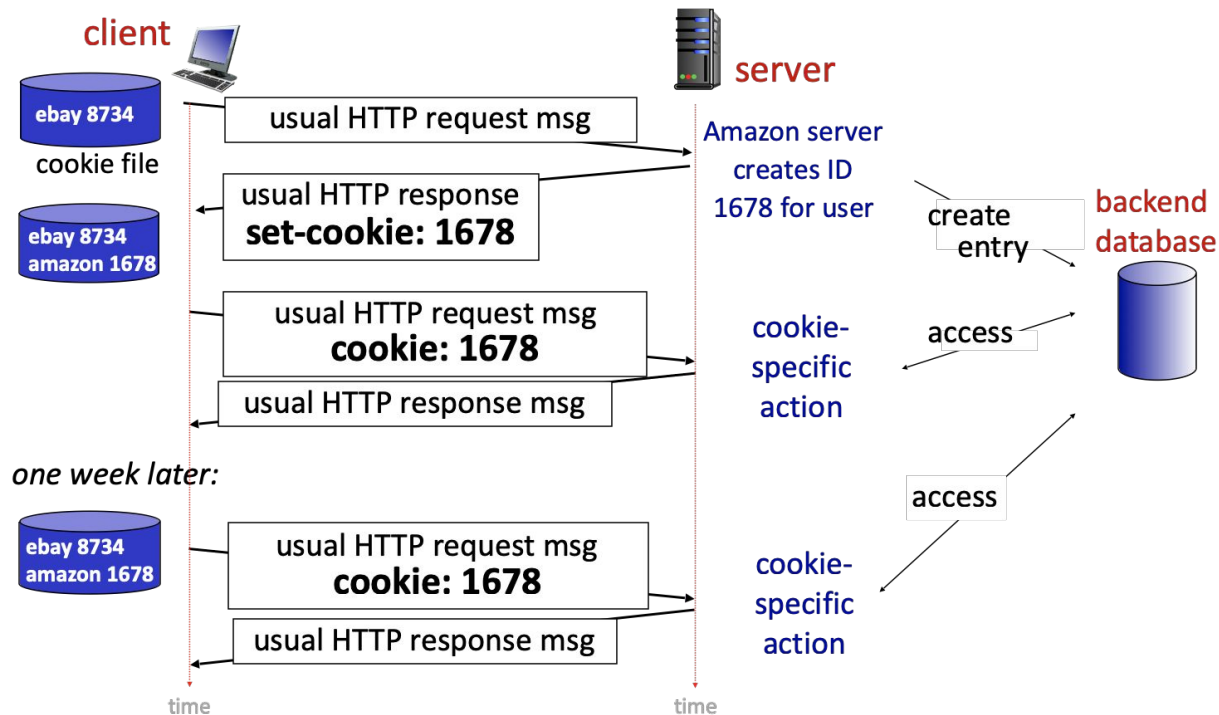


What happens if network connection or client crashes at t' ?

- HTTP is stateless
- **BUT**, web sites require to maintain state between transactions

Maintaining user server/state using COOKIES

Web sites & client browser use cookies to maintain state between transactions



four components:

- 1) cookie header line of HTTP response message
- 2) cookie header line in next HTTP request message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

HTTP cookies

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

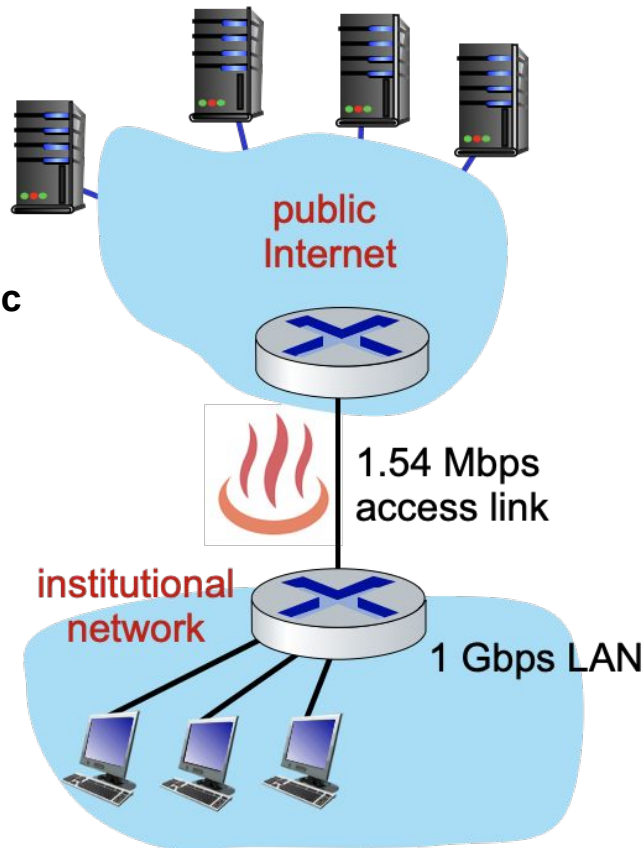
Why do we need web caches?

Example scenario

- access link rate: **1.54 Mbps**
- RTT from institutional router to server: **2 sec**
- web object size: **100K bits**
- average request rate from browsers to origin servers: **15/sec**
 - avg data rate to browsers: **1.50 Mbps**, (i.e., $15/\text{sec} * 100\text{Kb}$)

Performance:

- access link utilization = **0.97**
- LAN utilization = **0.0015**
- end-end delay = Internet delay +
access link delay + LAN delay
= **2 sec** + **minutes** + **usecs**



Solution 1: buy a faster access link

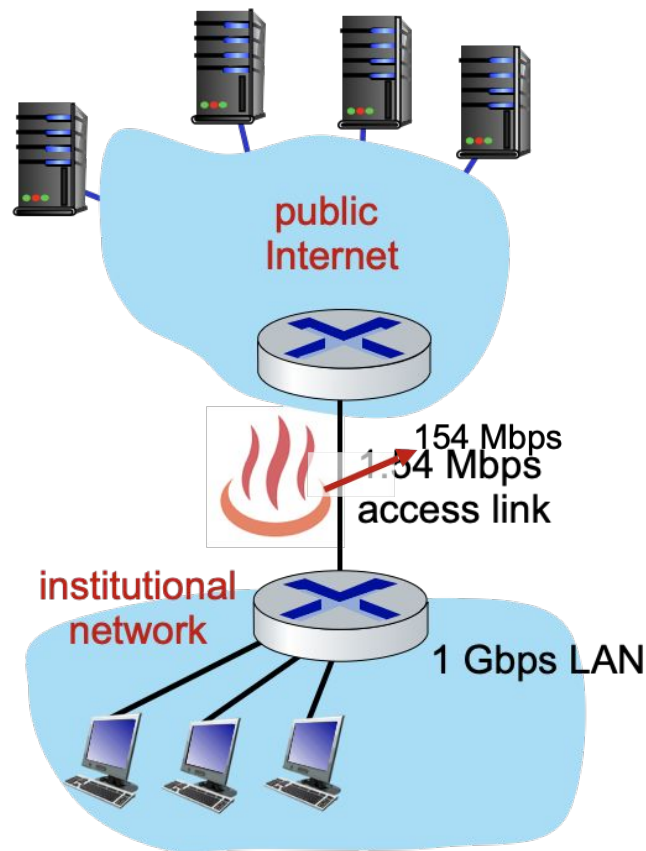
Example scenario

- access link rate: ~~1.54~~ **154 Mbps**
- RTT from institutional router to server: **2 sec**
- web object size: **100K bits**
- average request rate from browsers to origin servers: **15/sec**
 - avg data rate to browsers: **1.50 Mbps**

Performance:

- access link utilization = ~~0.97~~ **.0097**
- LAN utilization = 0.0015
- end-end delay = Internet delay +
access link delay + LAN delay
= **2 sec** + ~~minutes~~ **usecs**
msecs

Cost: faster access link (expensive!)



Solution 2: install a web cache

Example scenario

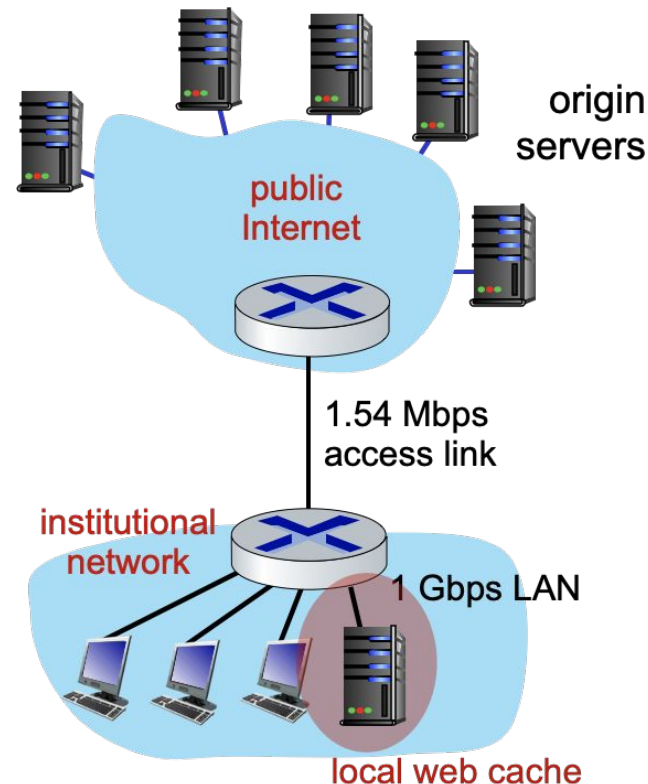
- access link rate: **1.54 Mbps**
- RTT from institutional router to server: **2 sec**
- web object size: **100K bits**
- average request rate from browsers to origin servers: **15/sec**
 - avg data rate to browsers: **1.50 Mbps**

Cost: web cache (cheap!)

Performance:

- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

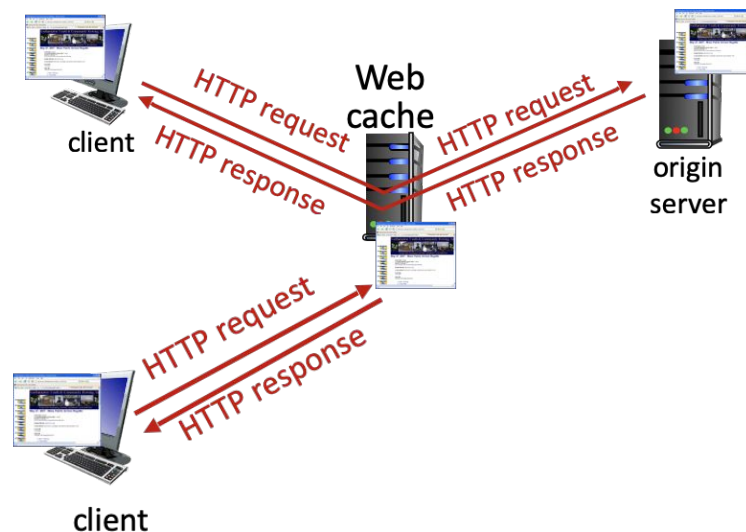
How to compute link utilization & delay?
Before this, what is a WEB CACHE??



Web caches

Goal: satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
 - *if* object in cache: cache returns object to client
 - *else* cache requests object from origin server, caches received object, then returns object to client



Web caches (a.k.a., proxy servers)

- Web cache acts as both client and server
 - server for original requesting client
 - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

Why Web caching?

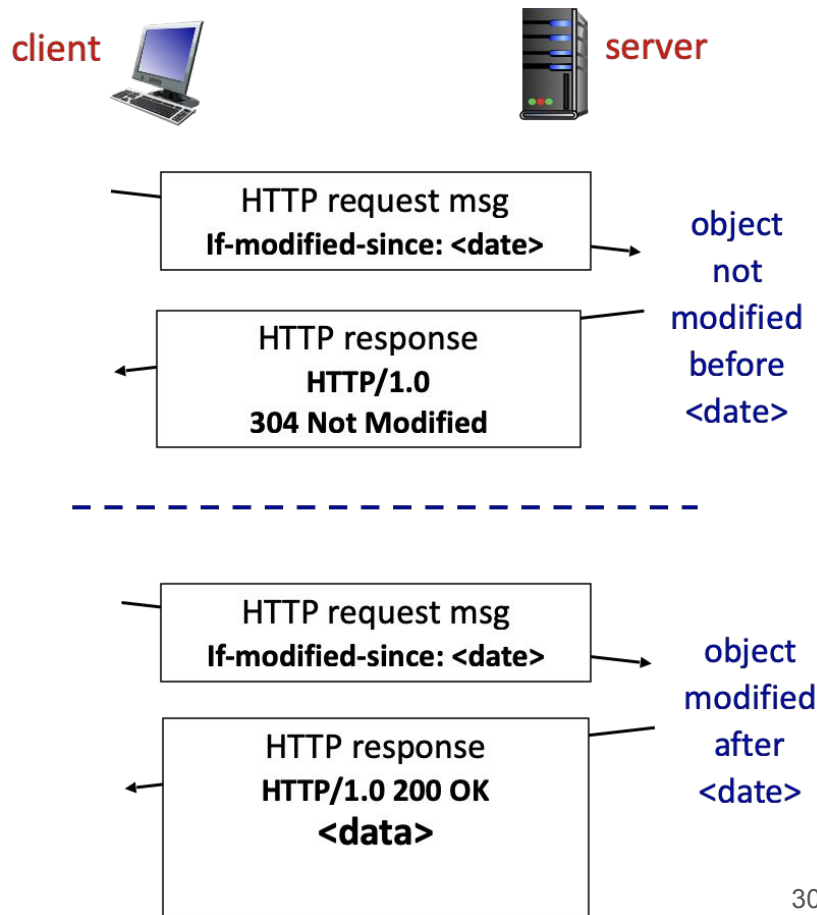
- reduce response time for client request
 - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
 - enables “poor” content providers to more effectively deliver content

Conditional GET

Goal: don't send object if cache has up-to-date cached version

- no object transmission delay (or use of network resources)

- ❑ **client:** specify date of cached copy in HTTP request
`If-modified-since: <date>`
- ❑ **server:** response contains no object if cached copy is up-to-date:
`HTTP/1.0 304 Not Modified`



RECAP: Solution 2: install a web cache

Example scenario

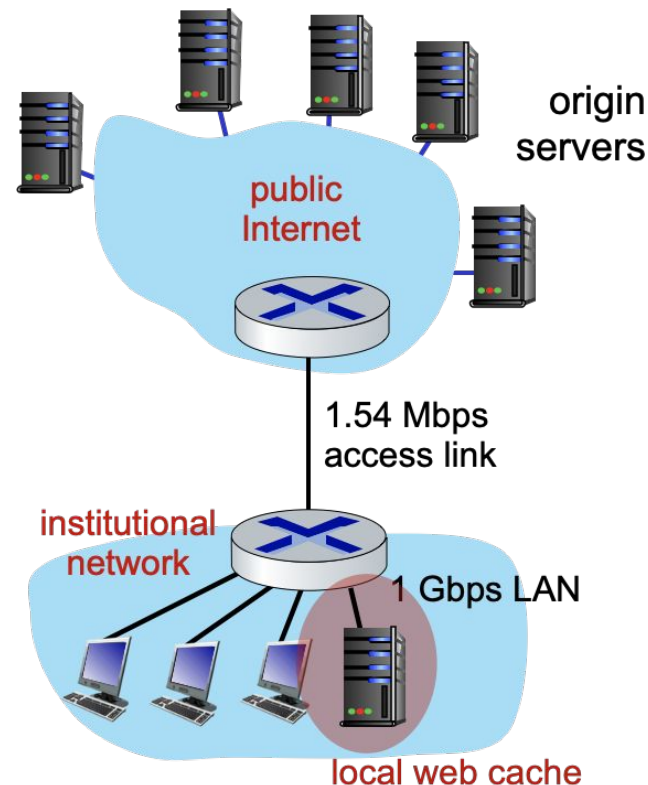
- access link rate: **1.54 Mbps**
- RTT from institutional router to server: **2 sec**
- web object size: **100K bits**
- average request rate from browsers to origin servers: **15/sec**
 - avg data rate to browsers: **1.50 Mbps**

Cost: web cache (cheap!)

Performance:

- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

How to compute link utilization & delay?
Before this, what is a WEB CACHE??



Calculating access link utilization, end-end delay with cache

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
 - rate to browsers over access link
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
 - access link utilization $= 0.9 / 1.54 = .58$ means low (msec) queueing delay at access link
- average end-end delay:
 - $= 0.6 * (\text{delay from origin servers})$
 - $+ 0.4 * (\text{delay when satisfied at cache})$
 - $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$

lower average end-end delay than with 154 Mbps link (and cheaper too!)

